

Multi-Tool Agentic Reasoning

1. The Agenda

In the previous session, we covered tool calling with a single tool. The model was able to call one tool when needed, like fetching the current time. That was a great start, but it was still close to a regular function call wrapped around an LLM.

In this session, the goal was to make a leap from single tool calling to true agentic behavior. We wanted the model to take a single user prompt, decide which tools are needed, call them in the right order, and produce one polished, combined answer.

What we set out to achieve

- Register multiple tools with a single ChatClient call.
- Let the model decide which tools to use based on the user's prompt.
- Add a system prompt to give the AI a clear role and direction.
- Demonstrate the model orchestrating multiple tool calls automatically.
- Build a clean, professional controller students can extend later.

End result

One endpoint that behaves like a real personal assistant. Same code structure, completely different intelligence.

2. What We Built

Three independent tools, each exposed to the LLM through the @Tool annotation, and a controller that wires them into the ChatClient.

Tool	Purpose	Input Parameter
DateTimeTool	Returns current date and time for any timezone	timeZone (Asia/Kolkata, etc.)
WeatherTool	Fetches real-time weather using wttr.in	location (Mumbai, Pune, etc.)
NewsTool	Pulls latest headlines using NewsAPI	topic (AI, Mumbai, cricket, etc.)

3. The Tools, Cleaned Up

DateTimeTool

Improved with proper formatting and a safe fallback to Asia/Kolkata when the timezone is invalid.

```
@Component
public class DateTimeTool {

    @Tool(description = "Get the current date and time for a specific timezone. "
+
        "Use this when the user asks about today, tomorrow, " +
        "current time, or date-related questions.")
    public String getCurrentDateTime(
        @ToolParam(description = "Timezone ID like Asia/Kolkata,
America/New_York, " +
        "Europe/London. Default to Asia/Kolkata if
not specified.")
        String timeZone) {

        System.out.println("DateTime Tool called for: " + timeZone);
        try {
            ZonedDateTime now = ZonedDateTime.now(ZoneId.of(timeZone));
            return now.format(DateTimeFormatter.ofPattern("EEEE, dd MMMM yyyy,
hh:mm a z"));
        } catch (Exception e) {
            return ZonedDateTime.now(ZoneId.of("Asia/Kolkata"))
                .format(DateTimeFormatter.ofPattern("EEEE, dd MMMM yyyy,
hh:mm a z"));
        }
    }
}
```

WeatherTool (new)

Uses the free wttr.in service. No API key needed, ideal for class demos.

```
@Component
public class WeatherTool {

    private final RestTemplate restTemplate = new RestTemplate();

    @Tool(description = "Get the current weather conditions for any city or
location.")
    public String getCurrentWeather(
        @ToolParam(description = "City name, for example Mumbai, Pune,
Bangalore")
        String location) {

        System.out.println("Weather Tool called for: " + location);
        try {
            String url = "https://wttr.in/" + location + "?format=3";
            return restTemplate.getForObject(url, String.class);
        } catch (Exception e) {
            return "Weather data not available for " + location + " right now.";
        }
    }
}
```

NewsTool

API key moved to **application.properties** using **@Value**. This is professional Spring practice.

```
@Component
public class NewsTool {

    private final RestTemplate restTemplate = new RestTemplate();

    @Value("${news.api.key}")
    private String apiKey;

    @Tool(description = "Get the latest news headlines and articles on any topic,
" +
                    "city, person, company, or event.")
    public String getLatestNews(
        @ToolParam(description = "Topic to search news for, like Mumbai, AI,
cricket")
        String topic) {

        System.out.println("News Tool called for: " + topic);
        try {
            String url = "https://newsapi.org/v2/everything?q=" + topic
                + "&pageSize=5&sortBy=publishedAt&apiKey=" + apiKey;
            return restTemplate.getForObject(url, String.class);
        } catch (Exception e) {
            return "News not available for " + topic + " right now.";
        }
    }
}
```

4. The Controller With Two Endpoints

Two endpoints serve different purposes. Both have access to all three tools, but they show two different ways of working with the model.

```
@RestController
public class Telusko {

    private ChatClient chatClient;

    @Autowired private DateTimeTool dateTimeTool;
    @Autowired private NewsTool newsTool;
    @Autowired private WeatherTool weatherTool;

    ChatMemory chatMemory = MessageWindowChatMemory.builder().build();
    String conversationId = UUID.randomUUID().toString();

    public Telusko(ChatClient.Builder builder) {
        this.chatClient = builder

.defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
        .defaultAdvisors(a -> a.param(ChatMemory.CONVERSATION_ID,
conversationId))
        .build();
    }

    // Endpoint 1: Tools registered, no system prompt
    @GetMapping("/info/{prompt}")
    public String getInfo(@PathVariable String prompt) {
        ChatResponse response = chatClient.prompt(prompt)
            .tools(dateTimeTool, weatherTool, newsTool)
            .call()
            .chatResponse();
        System.out.println("Model used: " + response.getMetadata().getModel());
        return response.getResult().getOutput().getText();
    }

    // Endpoint 2: Tools + system prompt = true assistant behavior
    @GetMapping("/assistant/plan")
    public String planMyDay(@RequestParam String prompt) {
        return chatClient.prompt(prompt)
            .system("""
                You are a smart personal assistant.
                You have access to tools for date and time, weather, and
latest news.

                Use whichever tools you need to answer the user clearly
and completely.
            """)
            .call()
            .chatResponse()
            .getResult()
            .getOutput()
            .getText();
    }
}
```

```

        Always give a helpful, well-organized response.
        """
    .tools(dateTimeTool, weatherTool, newsTool)
    .call()
    .content();
}
}

```

5. The Power of the System Prompt

The biggest difference between **/info** and **/assistant/plan** is the system prompt. The user never sees it, but the model reads it before every request and treats it as the rulebook.

Without system prompt

The model answers like a generic chatbot. It might use tools, might not, and the tone or structure of the response varies.

With system prompt

The model knows it is a personal assistant, knows what tools it has, and knows it should always give a well-organized response. Behavior becomes predictable, professional, and consistent.

Aspect	User Prompt	System Prompt
Changes per request	Yes, every time	No, stays the same
Visible to user	Yes	No, hidden
Defines	The question	The role and rules
Example	What is the weather in Pune?	You are a personal assistant. Use tools wisely.

6. How the Model Decides Which Tool to Call

This is the part that feels magical, but it is actually grounded in three things you control:

1. Tool name: A clear, descriptive name like `getCurrentWeather` tells the model exactly what the tool does.
2. Tool description: The `@Tool(description = "...")` annotation is the most important piece. The model reads this to decide when to invoke the tool.
3. Parameter descriptions: The `@ToolParam` annotation helps the model know what value to pass.

When a user asks a question, the model reads the user's intent, scans all available tool descriptions, and picks the right one. If multiple tools are relevant, it calls them in sequence and combines the results.

The hidden rule

Tools are only as good as their descriptions. A vague description leads to wrong tool selection or no selection at all. Write descriptions like API contracts: clear, specific, and unambiguous.

7. Postman Testing Guide

Three Postman requests for each endpoint, tested independently. Run them in order to see how the model adapts.

Endpoint 1: /info/{prompt}

Tools registered, no system prompt. Good for testing raw tool selection behavior.

Test 1: DateTime Tool

Method: GET

URL:

<http://localhost:8080/info/What is the current date and time in India?>

Expected Console Log:

DateTime Tool called for: Asia/Kolkata

Sample Response:

The current date and time in India is Saturday, 28 June 2026, 09:45 PM IST.

Test 2: Weather Tool

Method: GET

URL:

<http://localhost:8080/info/What is the current weather in Mumbai?>

Expected Console Log:

Weather Tool called for: Mumbai

Sample Response:

Mumbai: rainy +28C, light winds, 85% humidity.

Test 3: News Tool

Method: GET

URL:

<http://localhost:8080/info/Give me the latest news about Artificial Intelligence.>

Expected Console Log:

News Tool called for: Artificial Intelligence

Sample Response:

Top AI headlines: OpenAI launches new model, Google announces Gemini update, etc.

Endpoint 2: /assistant/plan

Tools plus system prompt. This is where the assistant behavior shines.

Test 4: DateTime via Assistant

Method: GET

URL:

<http://localhost:8080/assistant/plan?prompt=What is the current date and time in India?>

Expected Console Log:

DateTime Tool called for: Asia/Kolkata

Sample Response:

The current date and time in India is Saturday, 28 June 2026, 09:45 PM IST. Hope you are having a great day!

Test 5: Weather via Assistant

Method: GET

URL:

<http://localhost:8080/assistant/plan?prompt=What is the current weather in Mumbai?>

Expected Console Log:

Weather Tool called for: Mumbai

Sample Response:

Mumbai is currently experiencing rainy weather at +28C with 85% humidity. Carry an umbrella if you are heading out.

Test 6: All Tools via Assistant (THE MAGIC)

Method: GET

URL:

`http://localhost:8080/assistant/plan?prompt=I am visiting Pune tomorrow. Tell me the date, weather, and latest news.`

Expected Console Log:

`DateTime Tool called for: Asia/Kolkata Weather Tool called for: Pune News Tool called for: Pune`

Sample Response:

Travel briefing for Pune tomorrow (Sunday, 29 June 2026): expect rainy weather at +26C, carry an umbrella. Top local news includes road works on Sinhagad Road and a new tech park inauguration in Hinjewadi. Safe travels!

Pro Tip

Watch the console while running Test 6. Three Tool called for: lines from a single user prompt is the moment Agentic AI clicks. The model planned and orchestrated all of it on its own.

8. Why This Implementation Is Powerful

- **Zero hardcoded orchestration:** We did not write any code that says "call tool A, then tool B." The model decides everything.
- **Easy to extend:** Adding a fourth tool (email, calendar, database query) requires only a new `@Component` with `@Tool`. No controller changes needed except adding it to `.tools(...)`.
- **Same code for any LLM:** Switch from OpenAI to Gemini, Claude, or Ollama by changing properties. Tool calling behavior stays identical.
- **Production-ready foundation:** Memory advisor, conversation ID, dependency injection, clean separation of tools. Real applications are built exactly this way.
- **Honest behavior:** If the model does not have the right tool, it says so. No hallucinated data. This is critical for enterprise use.
- Wiring up observability with Grafana and Prometheus to monitor tool usage and token costs.